```
        repository.addBet(betId,walletId,marketId,odds,stake,result,session)
      case x =>                                                              ❺
        logger.debug("ignoring event {} in projection", x)

    }
  }
}
```

❶ Extends from JdbcHandler to write in the DB

❷ This session is loaded when JdbcProjection.exactlyOnce is created. More about this in the next listing.

❸ Events from Bet

❹ Passes Bet.Opened to the repository implementation

❺ Ignores any other event from Bet

Now you need to specify the source of the events: what kinds of events to feed into the projection. Here the source is an `EventSourcedProvider.eventsByTag[Bet .Event]`. This means it only picks up `Bet.Event` entries from the journal. You need `jdbc-connection-settings` in your configuration with the database connection details; you can reuse those connection details for writing, but you also need a database session, which here is a `ScalikeJdbcSession`. Finally, set the tags to provide parallelization. To review these ideas, see chapter 10, sections 10.2 and 10.3.

With the source provider, the handler, the repository implementation, and the JDBC connection, you can create the projection.

Listing 15.3 Basic components of the projection

```
object BetProjection {
  ...

  def createProjection(
      system: ActorSystem[_],
      repository: BetRepository,
      index: Int)
```